Meta-Learning with Muon: Learning the Dynamics and Geometry of Matrix Optimizers via Gradient Descent

Zacharie Bourlard Thomas Chetaille Marius Lhôte
École Polytechnique Fédérale de Lausanne (EPFL)

Abstract. Modern optimizers (e.g., AdamW, Muon, Newton–Muon) rely on fixed hyperparameters governing aspects such temporal dynamics (step size, inertia) and local geometry (curvature). We investigate whether both aspects can be meta-learned online using hypergradients [Chandra et al., 2022]. Applying this to the Muon family [Jordan et al., 2024, Du and Su, 2026] on a 51M-parameter GPT trained on WikiText, learning layerwise rates and momentum yields substantial gains (validation loss 3.47 vs. 4.46 for fixed Muon). Furthermore, while uniform Newton-style curvature proves fragile, softly gated geometric corrections (AdaGrad-EMA/Muon, SOAP-lite/Muon) achieve the best performance. Hypergradient meta-learning effectively transforms frozen optimizers into fully adaptive algorithms.

1. Introduction

Standard deep learning optimizers rely on hand-tuned hyperparameters—such as learning rates, momentum, and preconditioning strengths—that remain frozen during execution. However, the loss landscape evolves continuously, requiring adaptation in two key dimensions: the temporal dynamics (scale and inertia of steps) and the local geometry (curvature correction).

We investigate whether these optimization axes can be learned online by crossing two recent advancements. First, the hypergradient framework [Chandra et al., 2022] allows temporal hyperparameters to be optimized via automatic differentiation. Second, matrix-aware optimizers like *Muon* [Jordan et al., 2024, Liu et al., 2025] and *Newton–Muon* [Du and Su, 2026] have demonstrated significant empirical success over AdamW baselines, yet still depend heavily on rigid, hand-chosen structural coefficients.

In this work, we generalize hypergradient meta-learning to dynamically adapt both the time and geometry axes of matrix-aware optimizers. Our main contributions are:

- We introduce online, layerwise learning rates and momentum for Muon-style optimizers, yielding major stability and performance improvements.
- We extend meta-learning to the geometry axis, learning to dynamically gate when and where to inject curvature into Muon's orthogonalized update.
- We demonstrate that softly gated geometric corrections (e.g., AdaGrad-EMA/Muon, SOAP-lite/Muon) are highly robust, whereas uniform applications of Newton-style curvature remain fragile.

2. Background and Baselines

GD-UO: hypergradient meta-learning. After a weight update $w_t = w_{t-1} - \alpha \nabla_w \mathscr{L}_t(w_{t-1})$, the next loss can be differentiated with respect to α itself. Rather than deriving such hypergradients by hand, Chandra et al. [2022] keep α *attached* to the computation graph while detaching weight gradients (to avoid second-order terms) and previous weights (to bound the graph). The next `backward()` then automatically deposits:

$$h_t = \frac{\partial \mathscr{L}_{t+1}}{\partial \alpha_t}, \qquad \alpha_{t+1} = \alpha_t - \kappa h_t, \tag{1}$$

where κ is a hyper-learning-rate. Domain constraints use the following reparametrization: $\exp(\cdot)$ for positivity, $\sigma(\cdot)$ for $(0,1)$.

Muon and Newton–Muon. Muon [Jordan et al., 2024, Liu et al., 2025] accumulates Nesterov momentum $M_t = \mu M_{t-1} + g_t$ then orthogonalizes the update by approximating the factor $UV^\top$ with Newton–Schulz iterations:

$$X_0 = \frac{M_t}{\|M_t\|_F},$$
$$X_k = aX_{k-1} + b\,(X_{k-1}X_{k-1}^\top)X_{k-1} + c\,(X_{k-1}X_{k-1}^\top)^2 X_{k-1}, \tag{2}$$

converging to $X_N \approx UV^\top$ in N=5 steps. This equalizes all singular values of the update matrix: every direction in weight space receives an update of the same magnitude, regardless of how dominant or weak it was in the raw gradient.

Newton–Muon [Du and Su, 2026] motivates this from a curvature perspective. Consider a linear layer with weight matrix W_ℓ and input activations X_ℓ, so that

$$Y_\ell = W_\ell X_\ell. \tag{3}$$

Under a local quadratic model of the loss in the layer output, $\mathscr{L}(W_\ell) \approx \frac{1}{2}\|W_\ell X_\ell - Y_\ell^\star\|_F^2$, the gradient is

$$G_\ell = \nabla_{W_\ell}\mathscr{L} = (W_\ell X_\ell - Y_\ell^\star)X_\ell^\top, \tag{4}$$

and the input-side curvature is governed by the activation covariance $K_\ell = X_\ell X_\ell^\top$.

This shows why the geometry of the input activations matters. Plain gradient descent uses the same step scale in all directions, as if the local landscape were isotropic. When K_ℓ is highly anisotropic, the landscape seen by W_ℓ is closer to a narrow valley, or "gutter", than to a round bowl: updates may crawl along flat directions and become unstable along sharp ones. Muon partially addresses the problem by orthogonalizing matrix updates, imposing an equal-singular-value structure on the update. However, Muon does not explicitly correct the input-side geometry induced by X_ℓ.

Newton–Muon adds this input-side curvature information before the Muon projection:

$$\widetilde{G}_\ell = G_\ell\big(K_\ell + \lambda I\big)^{-1}, \qquad \Delta W_\ell = \mathrm{Muon}(\widetilde{G}_\ell), \tag{5}$$

where G_ℓ is the momentum-accumulated gradient, K_ℓ is a running estimate of the input-activation covariance, and λ is a ridge term for stability. Geometrically, the right-preconditioner attempts to turn the anisotropic gutter induced by correlated activations into a more isotropic bowl before applying Muon's matrix-normalized update.

In practice, however, the covariance estimate can be ill-conditioned, and the same amount of preconditioning is not necessarily useful for every layer or matrix type. This motivates our approach: instead of applying Newton-style preconditioning uniformly, we try to learn how strongly to inject it in each part of the network.

Reproduced baselines. To establish a solid foundation and reliable benchmarks, we reproduced the baseline results of Du and Su, 2026. On CIFAR-10 with a 32-layer ResMLP (width 512), we get Newton–Muon > Muon > AdamW (Table **??**). On language modeling (GPT 51M, WikiText, 2000 iters), we also reproduce those results. These results will serve as baseline to measure the results of our meta learning approach. Figure 1 shows both training curves. We will later focus only on the meta learning results of the GPT 51 model on wikitext as the dataset is bigger and it is more interesting to show improvements on the training of LLM and transofrmers, as a few pourcetange of improvements can make a huge differecne in the budgedt and computationnal ressources of the big trainings of LLm.

	CIFAR-10 (ResMLP-32)		WikiText (GPT 51M)	
Optimizer	**Final acc.**	**Best acc.**	**Val loss**	**Val acc.**
AdamW	60.51%	60.98%	4.472	0.2999
Muon	66.24%	66.39%	4.096	0.3412
Newton–Muon	**67.33%**	**67.49%**	**4.050**	**0.3444**

Table 1: Baseline comparisons at 2000 training steps. Newton–Muon denotes the best stable configuration. Results correspond to the baseline curves shown in Fig. 1.

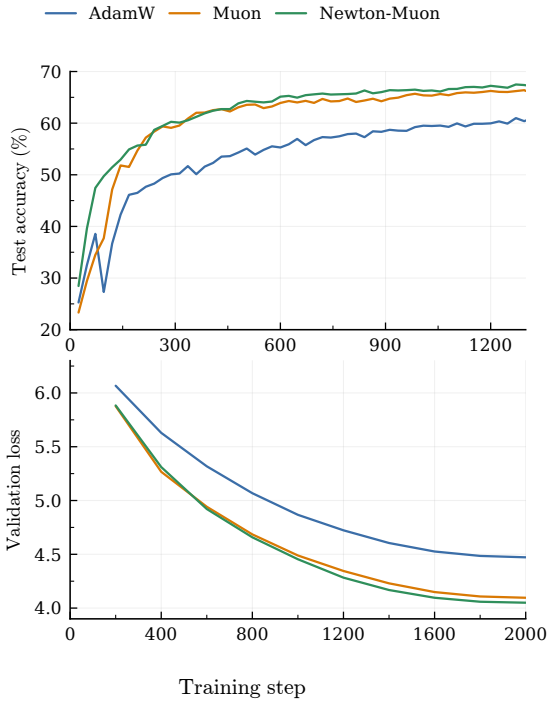

Figure 1: Baseline curves. *Top*: CIFAR-10 test accuracy (ResMLP-32). *Bottom*: WikiText validation loss (GPT 51M). Newton–Muon wins on CIFAR and WikiText.

3. Learning the Hyperparameters

3.1. Learning the dynamics (LR & momentum)

Method. Following Eq. (1), we keep α and μ attached to the computation graph during Muon's weight update: the learning rate α (reparametrized as $e^{\alpha_{\text{raw}}}$ for positivity) and momentum μ (reparametrized as $\sigma(\mu_{\text{raw}})$ for normalization) receive hypergradients automatically at the next backward pass. Weight gradients are detached to prevent second-order computation and hypergradients are clipped for stability. We test two granularities: (i) a single global LR shared by all layers; and (ii) a layerwise variant where each parameter group of each transformer block (query/key/value projections, attention output projection, MLP expansion, MLP output projection) learns its own LR scale and momentum.

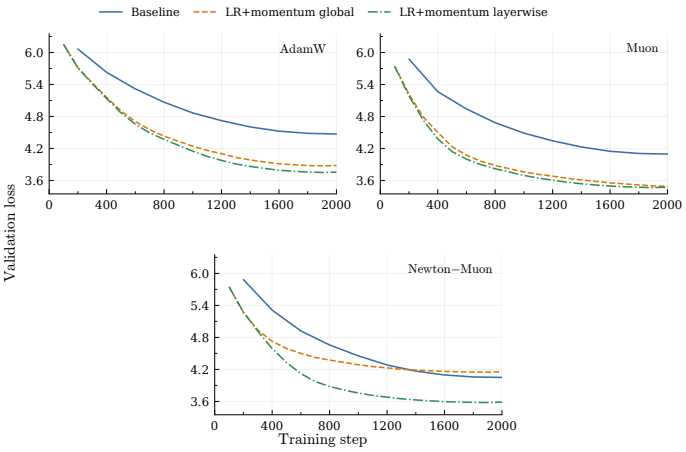

Figure 2: Learning the dynamics (WikiText, GPT 51M). Fixed baseline vs. global meta-learned) vs. layerwise meta-learned LR+momentum for AdamW, Muon, Newton–Muon. Muon-GDUO layerwise reaches best val loss **3.468**. One can note however that Adam has the smallest training time.

Results. Figure 2 compares the fixed baseline against the global and layerwise meta-learned variants for each base optimizer. We first observe that layerwise meta-learning consistently outperforms the fixed baselines across all three optimizers. AdamW remains the weakest optimizer in terms of final validation loss, but it is cheaper per iteration: the layerwise AdamW run takes about 2.19 s/it, compared with 2.30 s/it for Muon and 2.58 s/it for Newton–Muon. This is expected because AdamW only uses coordinate-wise adaptive statistics, while Muon requires matrix orthogonalization and Newton–Muon adds activation-aware preconditioning. Thus, AdamW is computationally attractive, but its cheaper steps do not compensate for its weaker optimization trajectory. While Newton–Muon is the strongest fixed baseline, Muon benefits more from layerwise meta-learning and ultimately achieves the best overall validation loss (**3.468** at 2000 iterations). This is likely because Newton–Muon's existing geometric corrections interact more complexly with dynamic hyperparameters, making it harder to tune. The per-layer trajectories (Fig. 3) reveal clear structural patterns. LR scales consistently grow with depth: early layers are heavily damped to preserve stable representations, while deeper, task-specific layers can afford larger steps. Momentum, however, remains remarkably stable across blocks, staying between roughly 0.945 and 0.975. This suggests that the amount of gradient smoothing needed for stable training is much more uniform than the optimal step size.

3.2. Learning the geometry (curvature)

Motivation and method. Learning the learning rate and momentum changes the time dynamics of the optimizer: how large each step is, and how much past gradients are reused. However, it does not directly change the geometry of the step. By geometry, we mean the direction in parameter space after accounting for local curvature or anisotropy. If the loss is much steeper in some directions than in others, the raw gradient can point in a poorly scaled direction.

Newton–Muon tries to fix this by preconditioning the gradient with information from the input activations. In practice, applying this correction uniformly can be unstable: some parameter groups benefit from curvature information, while others are

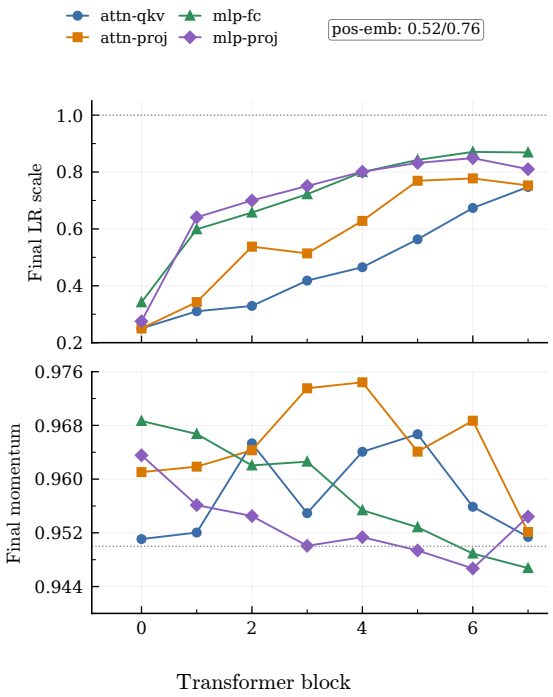

Figure 3: Learned layerwise hyperparameters (Muon-GDUO, GPT 51M, WikiText, 2000 iters). *Top*: final learning-rate scale per transformer block and parameter group. *Bottom*: final momentum. The dotted lines mark the starting values. LR scales grow with depth and differ across groups, query/key/value projections are consistently the most damped.

harmed by it. We therefore do not replace Muon by a preconditioned optimizer. Instead, we keep Muon as the backbone and learn a small residual *gate* that decides how much curvature correction to inject.

We use the following residual gating rule:

$$g_{\text{geo}} = g + s_{\text{bucket}}\big(P(g) - g\big), \quad \Delta W = \text{Muon}\big(\text{momentum}(g_{\text{geo}})\big).$$

Here $P(\cdot)$ proposes a geometry-aware version of the gradient, and s_{bucket} is a learned gate strength. If $s_{\text{bucket}} = 0$, the method is exactly Muon. If $s_{\text{bucket}} = 1$, the bucket fully uses the preconditioned gradient before Muon's projection. Intermediate values let the optimizer interpolate between the two. The gate is learned separately for each parameter bucket, such as query/key/value projections, attention output projections, MLP expansion matrices, and MLP output matrices.

This makes the curvature correction conservative: the preconditioner can help only where the hypergradient shows that it improves training. Physically, the gate learns *where* the geometry estimate is useful and *how strongly* it should modify Muon's direction.

We test four choices of P: Adam/Muon gives a coordinate-wise adaptive direction, Muon/Newton corrects for anisotropy in the input activations, AdaGrad-EMA/Muon uses a stable diagonal estimate of repeated gradient energy, and SOAP-lite/Muon uses a lightweight matrix preconditioner to correct correlations across rows and columns.

Results. Table 2 reports the 2000-iteration WikiText results. The main pattern is clear: soft preconditioning works better than hard Newton-style corrections. Adam/Muon, AdaGrad-EMA/Muon, and SOAP-lite/Muon all improve over the Muon LR+momentum layerwise reference. AdaGrad-EMA/ Muon gives the best validation loss and accuracy in this sweep, while SOAP-lite/Muon is very close. Runtime is also informative. Adam/Muon is slightly faster per iteration than the Muon layerwise reference, which is expected because the Adam-style correction is cheap and does not require matrix preconditioning. This matters: a geometry-aware variant is only useful in practice if its gain in loss is not erased by a slower step time.

In contrast, SOAP-lite and Muon/Newton add extra preconditioning cost, so their accuracy gains must be interpreted together with their seconds per iteration. Muon/Newton is worse. Our logs suggest that the Newton correction is not always a useful new direction: in several buckets it is highly aligned with the Muon update, so it mostly rescales the step, while in other buckets it can become unstable and require clipping. This supports the central design choice: curvature should not be injected everywhere with a fixed strength. It should be weak, bucket-dependent, and learned online before Muon's final projection.

Variant	Setting	Val loss	Acc.	s/it
Adam/Muon	residual gate	3.409	0.3989	**2.22**
Muon/Newton	learned Newton strength	3.508	0.3885	2.35
AdaGrad-EMA/Muon	diagonal EMA precond.	**3.402**	**0.4005**	2.30
SOAP-lite/Muon	matrix precond.	3.411	0.3993	2.30
Meta-Muon (best model)	LR+momentum layerwise	3.477	0.3926	2.30

Table 2: Geometry learning on WikiText with a GPT 51M model after 2000 training iterations. Learned soft preconditioners and Adam/Muon gating improve over the Muon LR+momentum layerwise reference, while the Muon/Newton gate remains less stable. The diagonal AdaGrad-EMA preconditioner achieves the best validation loss and accuracy in this sweep. It is interesting to note that Adam/Muon has the smallest second per iteration ratio. Fig. 4 in Appendix represents the evolution of these variants accross steps.

4. Conclusion

The main goal of this project was to transform traditional, frozen optimizers into adaptive algorithms capable of adjusting themselves automatically during training, thereby reducing the need for expensive manual tuning. To achieve this, we explored two distinct optimization axes: temporal dynamics and local geometry.

First, regarding temporal dynamics, dynamically adapting layerwise learning rates and momentum established a strong baseline, reaching a **validation loss of 3.477** . This approach revealed an interpretable structure where early layers require damped updates to preserve foundational representations, while deeper layers can safely afford more aggressive steps.

Second, for the local geometry, we found that applying dense Newton preconditioning uniformly was fragile. Instead, introducing a learned residual gate allowed the optimizer to inject curvature corrections. This softly applied curvature turned out to be our most successful strategy, with the **diagonal AdaGrad-EMA/Muon configuration achieving the overall winning validation loss of 3.402**, outperforming both the temporal baseline and rigid Newton corrections.

Ultimately, hypergradient meta-learning successfully turns static, hand-tuned optimizers into fully dynamic systems. By adapting both time and geometry, these methods offer a highly promising path to significantly reduce the massive computational budgets typically required to tune large-scale models.

References

K. Chandra, A. Xie, J. Ragan-Kelley, and E. Meijer. Gradient descent: The ultimate optimizer. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/36ce475705c1dc6c50a5956cedff3d01-Abstract-Conference.html.

Z. Du and W. Su. The newton-muon optimizer, 2026. URL https://arxiv.org/abs/2604.01472.

K. Jordan et al. Muon: An optimizer for hidden layers in neural networks. Blog post, 2024. URL https://kellerjordan.github.io/posts/muon/.

J. Liu et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025. URL https://arxiv.org/abs/2502.16982.

Appendix

A. Run details

All language-modeling runs use a 51M-parameter GPT (8 transformer blocks) trained on WikiText. Baselines and geometry-gate runs: 2000 iterations. Layerwise dynamics run: 2000 iterations. Hypergradients are clipped; LR is parametrized in log-space; momentum via sigmoid, following the domain-constraint recommendations of Chandra et al. [2022]. Parameter buckets: query/key/value projections, attention output projection, MLP expansion, MLP output projection; embeddings and head handled separately or excluded depending on the run.

B. Additionnal plot

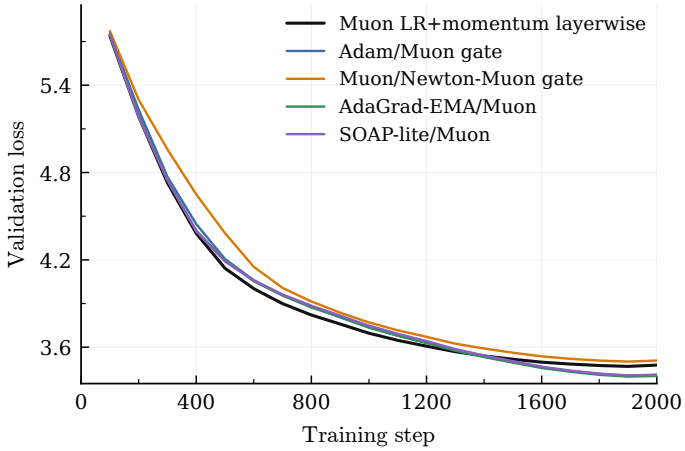

Figure 4: Learning geometry through gated preconditioning (GPT 51M, WikiText, 2000 iters). We compare several geometry-aware variants against the best temporal baseline, Muon with learned layerwise learning rate and momentum. Adam/Muon, AdaGrad-EMA/Muon, and SOAP-lite/Muon improve over the Muon baseline, showing that useful curvature corrections can be learned online. The Muon/Newton gate is less stable, suggesting that activation-based Newton preconditioning should be injected selectively rather than applied uniformly.